AI 利用時の注意点とコツ

2026-11-13

AI 利用時の注意点とコツ

基本方針・限界

- AI の出力をそのまま信頼せず、判断と検証は人が行う。
- 変更は少しずつ、結果を判断できる範囲でする。
- 知見の少ない分野では、入門資料の整理や確認項目の洗い出しに使える一方、自分だけで正誤を判断しにくい。重要な結論は一次資料・実物・専門家で確かめる。
- APIや製品仕様は、対象版の公式資料と実際の環境で確認する。検索・資料参照ができる場合も、古い情報や別版を混ぜていないかを見る。
- 新規ツールやフレームワークでの開発への適用は慎重にする。
- AI は与えられた資料やコードから一定の理解や推論はできるが、正しく理解したかの確認が必要。

指示・プロンプト

- 指示が曖昧だとぬけが多くなる。
- 意図通りにならなかったときは、期待結果と実際の差を示し、確認済みの原因と仮説、次の確認方法を分けて提示させる。AI自身が説明した原因も、ログや差分などで確かめる。
- バグや修正はできるだけ具体的に指摘するとよい。
- 操作対象が具体的でないと対象外のものも変更することがある。
- 対象や完了条件が不明、または影響が大きい作業では、実施内容を先に示させる。対象・制約が明確で承認済みの通常作業まで、毎回の確認待ちにする必要はない。
- 作業内容がどうなるか説明させ、確認してから実装を指示すると間違いを減らせる。
- 質問だけをしたいときは、「まだ実行しないで、どうなりますか」と明示する。
- 先にどうなるか見せてもらい、修正や追加をしながら仕上げる。

品質・レビュー

- 自分の考えと一致する返答も、正しさの証拠にはしない。同意・反対のいずれも根拠を確認し、返答への納得と検証結果を区別する。
- 不用なコードを生成することがあるため、チェックは必須。
- バグを作り込むことがあるため、リグレッションテストは必要。
- コード作成や修正のボリュームが多いと、レビューせずに受け入れてしまいやすい。
- レビューとチェックは誤りを見つける手段になるが、見落としがなくなる保証ではない。何をどこまで確認したかを残す。

- テストの負荷は、単純作業では減ることもあるが、新規実装や広範囲変更では増えやすい。
- 期待結果の根拠となる仕様・入力例を渡し、実装との差と未確認事項をレビューさせる。生成コードだけから期待値を作ったテストで、仕様への適合を確認したことにしない。
- 実装後は影響に応じたレビューとテストを行う。リファクタリングは必要性と影響を確認して行い、毎回必須とはしない。

デザイン・UI

- デザインは画像で指示するとよい。
- 描画の問題は画像で指摘するとよい。
- デザインを過剰変更してしまうことがある。
- 指示していない機能追加やデザイン変更は、実装前に提案として示させる。よい案でも、採用判断と実装は分ける。

ドキュメント

- ドキュメントを生成させても構成や説明順の調整が必要。
- ドキュメント修正では、どこをどのように修正するかを先に見せてもらうように指示するとよい。

AGENTS.md・運用

- 共通ルールとプロジェクト固有のルールを分け、利用するAIツールが実際に読み込む場所と適用範囲を確認する。例えばCodexの標準的な共通ルールの配置先は `~/.codex/AGENTS.md` であり、単にホーム直下へ置くこととは区別する。
- AGENTS.md の書き方や運用例を調べて読むとよい。
- 何でも AGENTS.md に書くのではなく、作業に適したファイルにするとよい。 例: 継続するルールは `AGENTS.md` 、現在地と次の作業は `NEXT.md` 、判断理由は `DECISIONS.md` 、長期の残作業は `backlog.md` に分ける。この3ファイルは独自の運用であり、ファイル名だけで自動読込されるとは限らない。

得意な作業・特性

- 「何々と同じように」は、意図通りで一貫性のある結果につながることがある。
- 多数のファイルの修正候補や記述の不一致を抽出する作業に利用できる。対象一覧と変更差分を照合し、処理漏れや対象外の変更を確認する。
- クラスの分離やライブラリー化にも利用できるが、依存関係・所有権・公開API・ビルド構成を変えるため、単純な作業と決めつけない。
- ただし、設計判断が絡む分離や公開 API 設計では、人の確認が必要。

実装方針・デバッグ

- 実装する前に、実装方針を確認してから実装を進める。
- 実装方針がよくないと思ったら、別の実装方針を考えさせるか、実装方針を提案する。
- なかなか実装がうまく行かないときには、今までの実装方針を提示させて検討する。
- なかなか実装がうまく行かないときには、実装案の得失を調べさせる。
- なかなか実装がうまく行かないときには、実装案を提案する。
- 実装を任せっきりにしない。こちらからも提案する。
- 実装検討はファイルに書き出して、やりとりすると効率的。 検討結果をファイルに書き出す -> レビューを書き込む -> 再検討...

コンテキスト復元

- 作業の区切りでは、確認済みの状態・次の作業・未検証事項を `NEXT.md`、判断理由を `DECISIONS.md`、長期の残作業を `backlog.md` に残す。
- 再開時は「NEXT.mdを読んでください」のように対象を指定し、実際に読めた内容、参照できない資料、次の作業を返してもらう。読込方法は製品や環境で異なるため、特定のコマンドを共通手順としない。
- 記録は会話の完全な復元を保証しない。現在のファイル・Git差分・実行結果とも照合する。ワークフローの旧版を履歴との対応のために保持している場合は、単純な更新漏れとして置き換えない。

製品固有の記述の参照先

- OpenAI公式: AGENTS.mdの配置と読み込み

製品固有の説明は2026年11月13日に確認したものです。一般的な実践案が、すべてのモデルや作業で効果検証済みという意味ではありません。